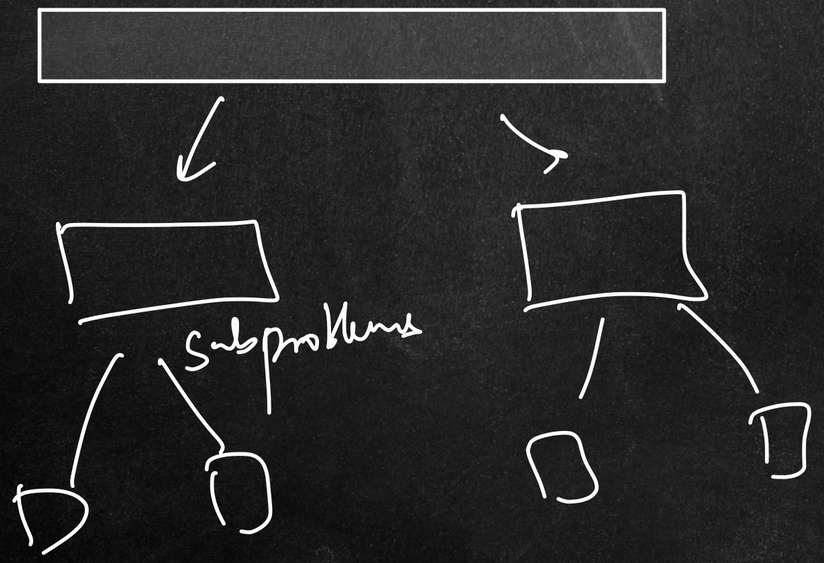
Reinforcement Learning

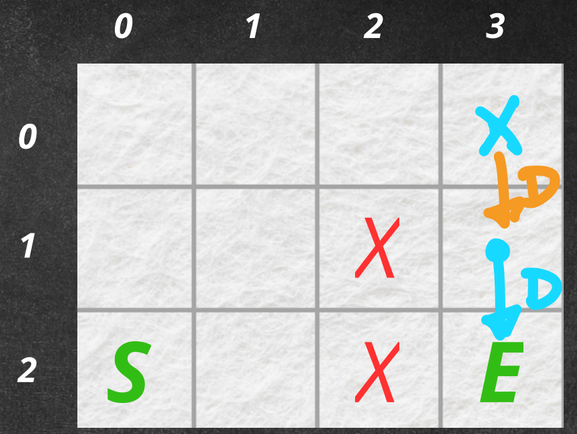
Dynamic Programming

VF / policy

Optimal subproblems

1. DP → DSA
2. Monte Carlo Methods
3. TD learning





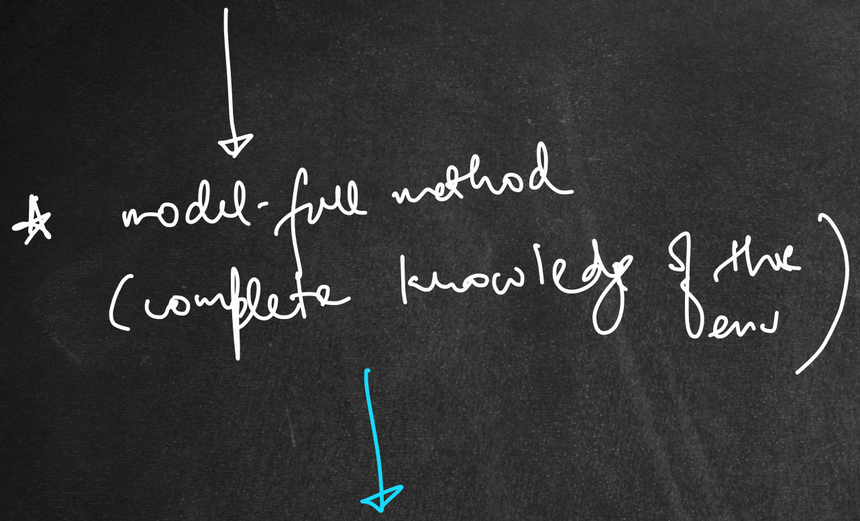
longest problem

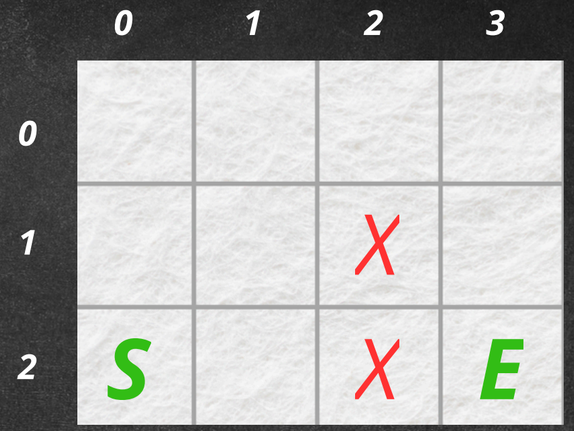
↓

smallest problem

Reinforcement Learning

Dynamic Programming

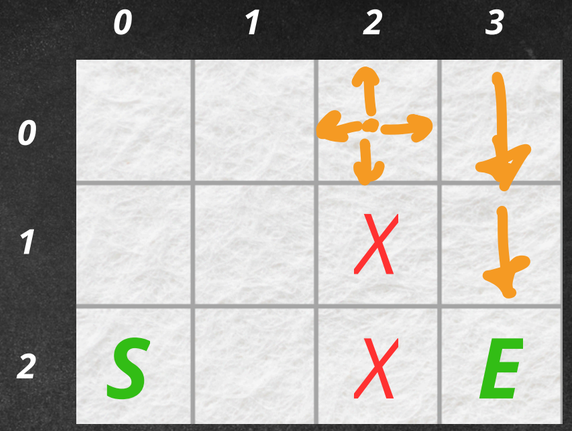




Reinforcement Learning

Dynamic Programming

$$V(s) = \max_a \left(R(s,a) + \gamma \cdot V(s') \right)$$



Disadv.

1. model-full
2. scalability
3. computationally expensive for large MDPs

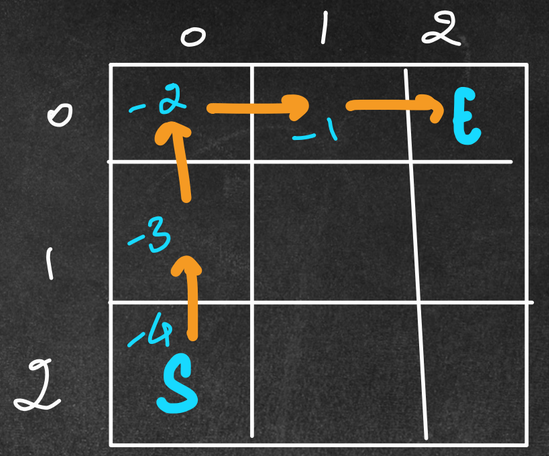
Reinforcement Learning

Monte Carlo Methods

↓

model-free algos

episodes → experienced samples



updation step

$V(2,0) = -4$

$V(1,0) = -3$

$V(0,0) = -2$

⋮

1st ep

step	state	action	reward
1	2,0	↑	-1
2	1,0	↑	-1
3	0,0	→	-1
4	0,1	→	0

$\gamma = 1$

$$G_t = R_{t+1} + \gamma^1 \cdot R_{t+2} + \gamma^2 \cdot R_{t+3}$$

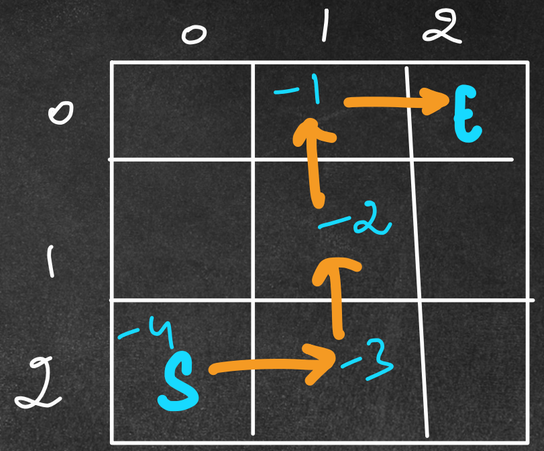
$$G_t = -1 + (-1) + -1 = -3$$

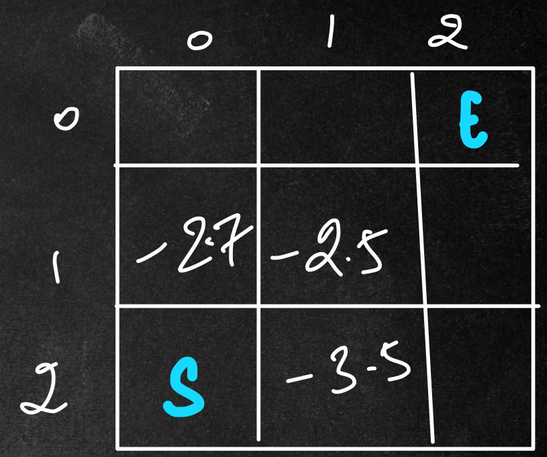
Reinforcement Learning

Monte Carlo Methods

eg2

step	state	action	reward
1	(2,0)	→	-1
2	2,1	↑	-1
3	1,1	↑	-1
4	0,1	→	-1





$V(2,0) = \frac{-4+(-4)}{2} = -4$

$V(2,1) = \frac{0+(-3)}{2} = -1.5$

⋮ ⋮ ⋮

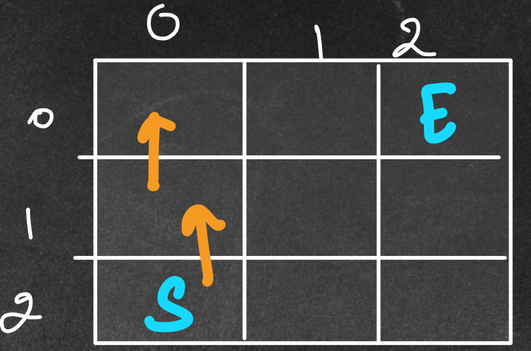
Reinforcement Learning

Monte Carlo Methods

1. episode based → long episodes → slow learning
2. continuous problems → MCM ✗
3. learning → avg. consequences of individual path → episode avg. consequence
   - → each step

Reinforcement Learning

Temporal Difference (TD) Learning



V(s)

0	0	0
0	0	0
-0.5	0	0

↓ model-free (steps)

updation rules

step 1

learning rate

$$V(S) = V(S) + \alpha.[R + \gamma.V(S') - V(S)]$$

TD

TD error

reward

statval of next state

statval of current state

$\alpha = 0.5$, $\gamma = 1$

$$V(2,0) = V(2,0) + 0.5[-1 + 1.V(1,0) - V(2,0)]$$

(V(2,0) = 0; V(1,0) = 0; V(2,0) = 0)

$$V(S) = 0.5.(-1) = -0.5$$

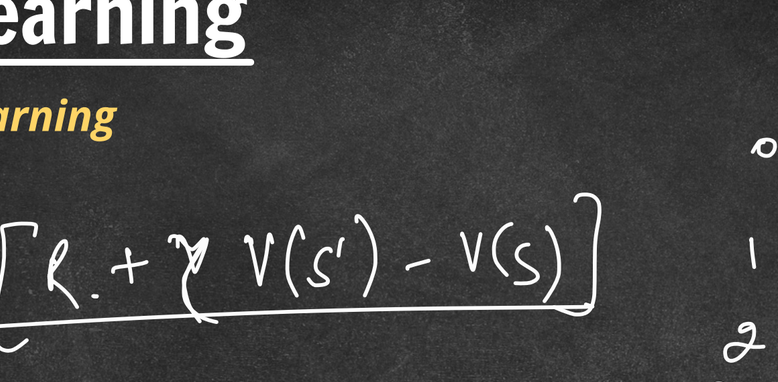
Reinforcement Learning

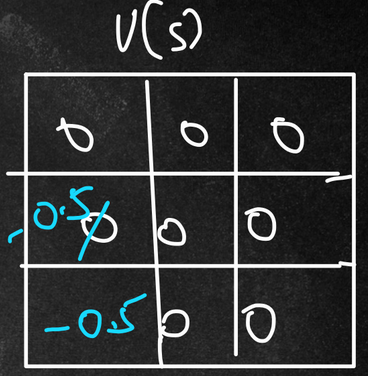
Temporal Difference (TD) Learning

$$V(s) = V(s) + \alpha \left[R + \gamma V(s') - V(s) \right]$$

$$V(1,0) = V(0,0) + 0.5\left[-1 + 1.\,V(0,0) - V(1,0)\right]$$

$$V(1,0) = -0.5$$





① SARSA

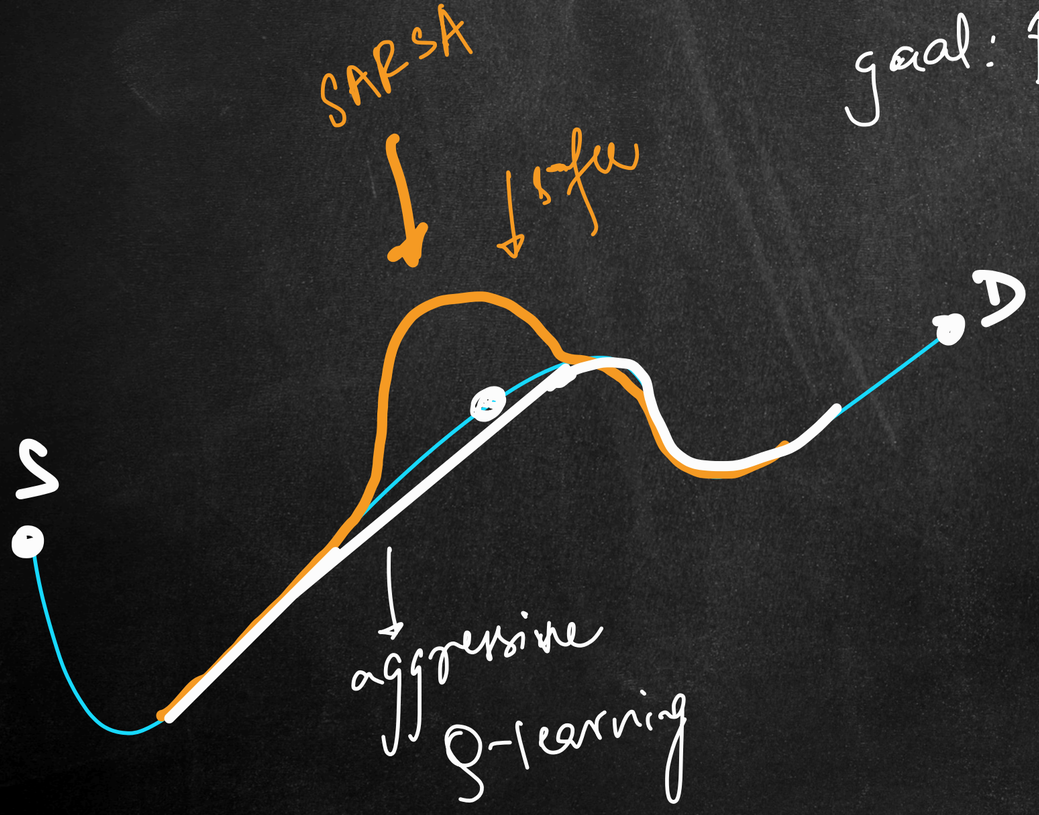
② Q-learning

① step wise learning

faster learning ⇒ faster convergence

Reinforcement Learning

SARSA



learn to act in an env
goal: ↑ total rewards

① SARSA
② Q-learning
> RL

① TD methods learning
② model free

③ Q-value or Action value

Reinforcement Learning → on-policy algo

SARSA → State, Action, Reward, Next State, Next Action

Q (state, action)

1. current state (S_t)
2. " action (A_t) ⇒ Policy (ε-greedy)
3. Take action
   - Reward → R_{t+1}
   - Nextstate → S_{t+1}
4. Next action (A_{t+1}) → Policy (S_{t+1}, A_{t+1})
5. updating Q-value $Q(s,a)$
   # SARSA update

1000 episodes

10% explore
90% exploid

$$Q(s,a) = Q(s,a) + \alpha\left[R + \gamma \cdot Q(s',a') - Q(s,a)\right]$$

learning rate

gamma

Reinforcement Learning

Q-Learning → off-policy

shortest path

1. curr state S_t
2. " action A_t → policy (ε-greedy)
3. Take action
   Reward R_{t+1}
   next state S_{t+1}
4. next action (A_{t+1}) ⇒ on the basis of Q-val next best action pick for next state
5. update Q-val # Q-learning update

optimal Q-val

$$Q(s,a) = Q(s,a) + \alpha \cdot \left[R + \gamma \left(\max Q(s',a) \right) - Q(s,a) \right]$$

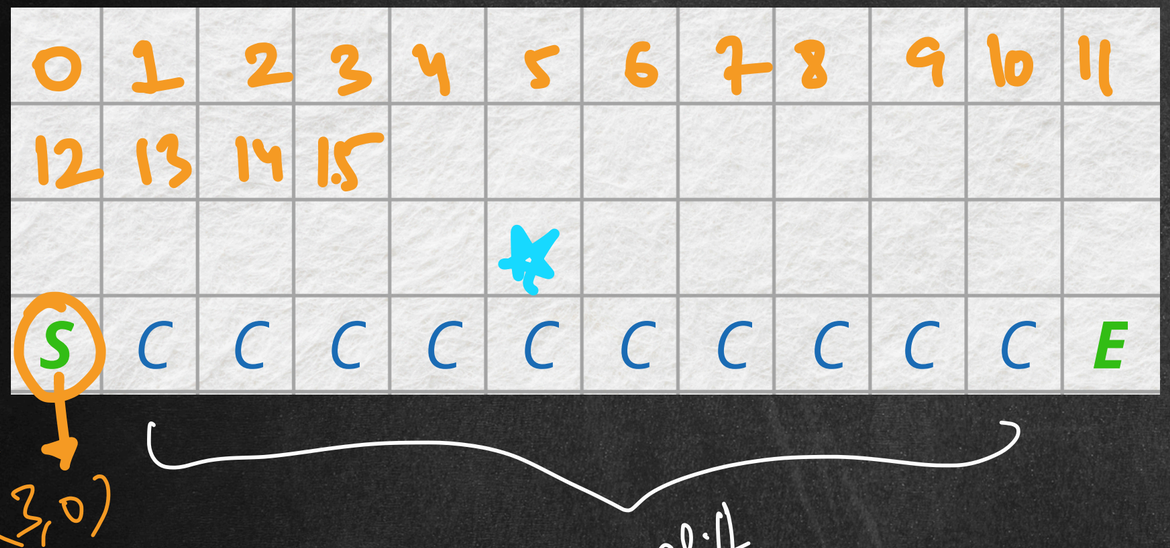
Reinforcement Learning

Cliff Walking Problem ⟶

env : 4x12 grid

Actions : ↑, ↓, ←, → {0, 3}

Reward : normal cells → -1
target " → 0
cliff → -100



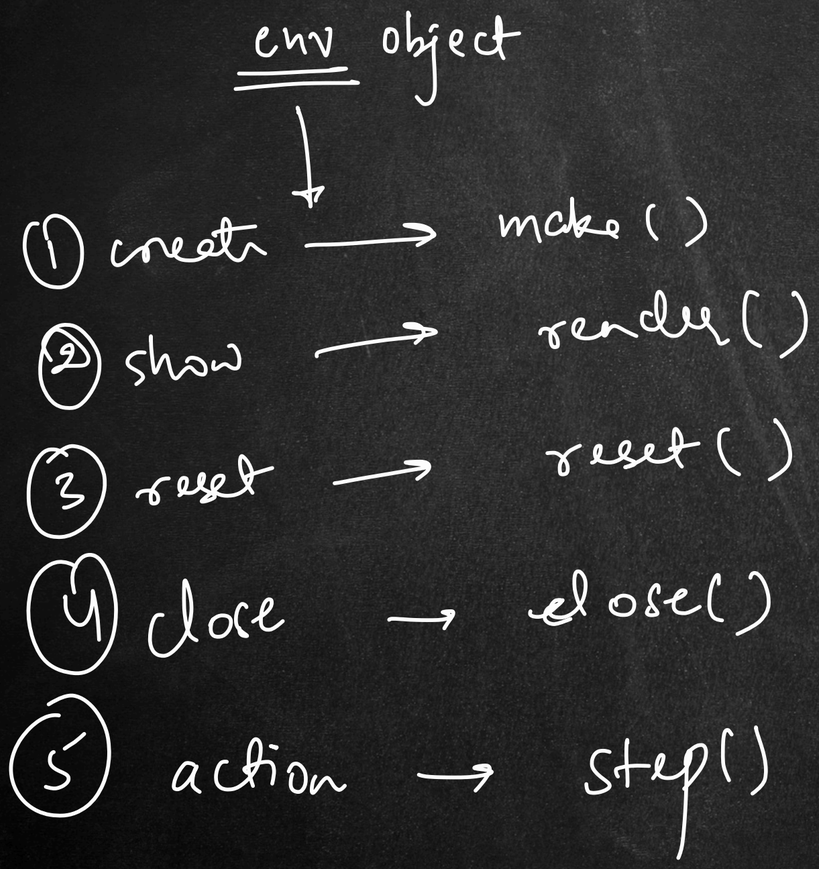
State : 4*12 = 48 cells (row, col)

(single num) ⟶ state = row *12 + col

state of S = 3*12 + 0 = 36

Reinforcement Learning

Gymnasium



Reinforcement Learning

SARSA Implementation

$$Q(s,a) = Q(s,a) + \alpha \cdot [R + \gamma \cdot Q(s',a') - Q(s,a)]$$

alpha

gamma

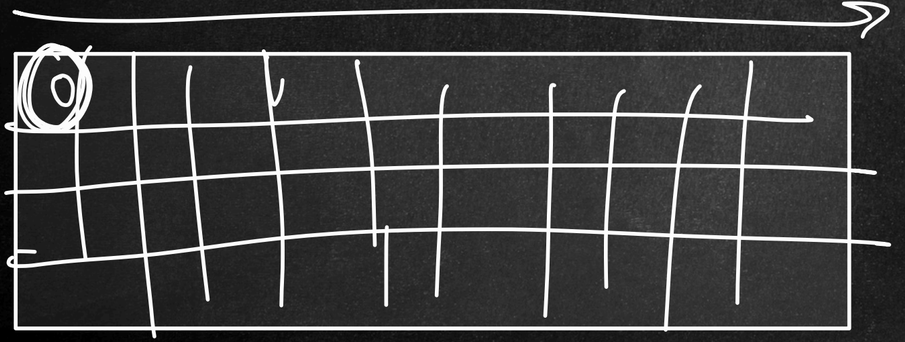
episodes = 500

Policy → ϵ greedy

Q-value → 48 x 4

States = 48

action = 4



1. alpha = 0.5
2. gamma = 0.99
3. epsidos = 500
4. epsilon = 0.1

Reinforcement Learning

SARSA Implementation

gym → humans

training → slow

done { terminate → agent reached E
truncate → cut short

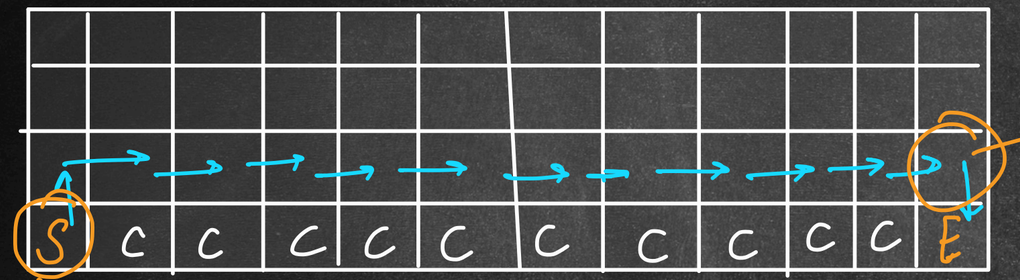
max time
" path length

Reinforcement Learning

Q-learning Implementation

$$Q(s,a) \mathrel{+}= \alpha * [R + \gamma \cdot \max Q(s',a) - Q(s,a)]$$

optimal Q-value



state = 35
action = down ↓ (2)

2,11

Q-table

state = 36
action = up ↑
(0)

$2 * 12 + 11$
$= 24 + 11$
$= 35$